CHECKPOINT

A puzzle game about quantum error detection, scored with Qiskit Aer and qiskit-paulice

> Checkpoint is a puzzle game in which the player protects noisy quantum circuits with coherent Pauli checks. Each placement is a real spacetime-code decision, and every score (post-selected fidelity, the fraction of runs kept, and the added hardware) is computed with Qiskit. This report covers the problem, how the game's rules map onto the physics, the brief's experiment, and five findings backed by the computed numbers. Everything can be reproduced from the repository.

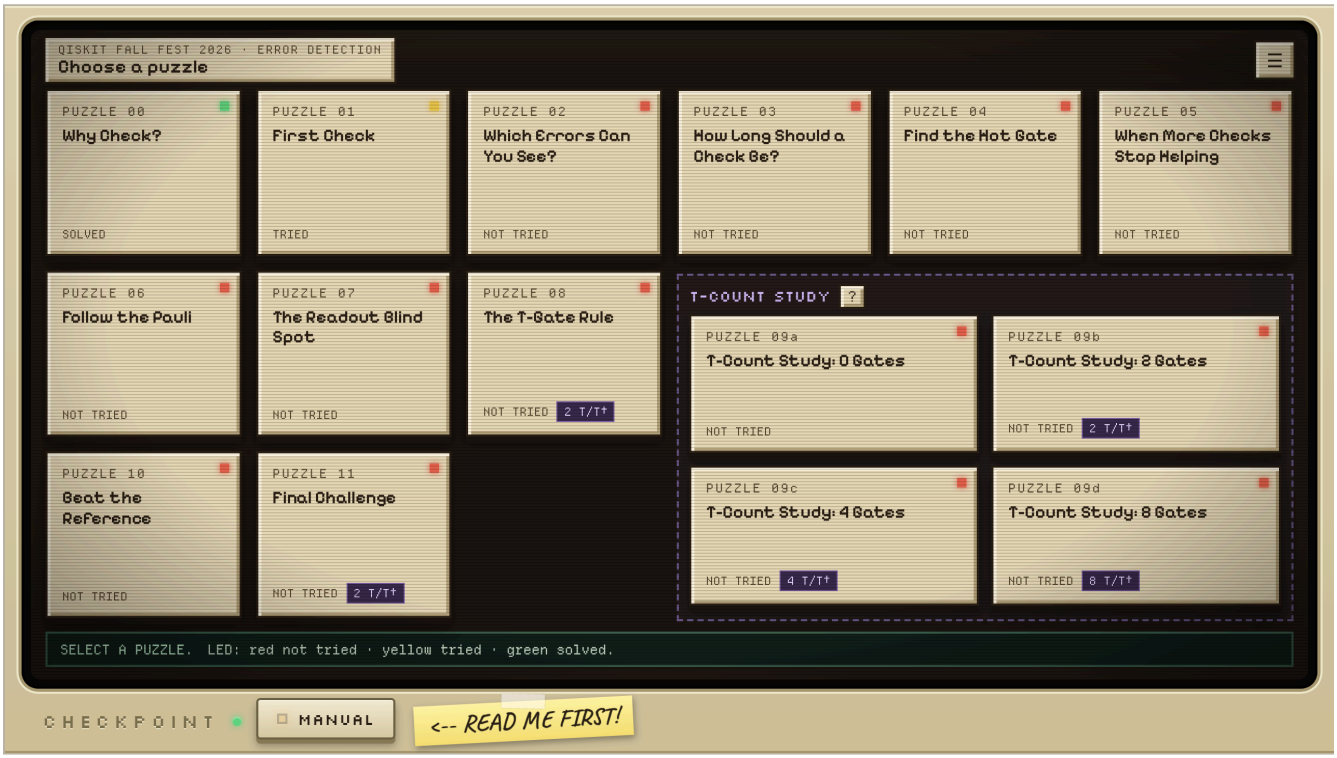


Figure 1. The puzzle list: 15 puzzles, including the four-part T-count study. LEDs show solved, tried and untried puzzles. The MANUAL button on the monitor opens the user manual; until it has been opened once, its light blinks and a sticky note points at it.

1 Problem

Quantum gates and measurements fail with small probabilities, and failures change the output. There are three broad responses:

- **Detection** (this project): add checks that flag some errors, and discard the flagged runs. Nothing is repaired. The kept runs are more often correct, but fewer runs are kept.
- **Mitigation**: keep every run and correct the estimate afterwards (zero-noise extrapolation, probabilistic error cancellation, readout correction).
- **Correction**: encode logical qubits, measure stabilizers repeatedly and undo errors during the computation.

A **spacetime check** (coherent Pauli check) watches a stretch of a circuit. An ancilla starts in $|+\rangle$ and applies a controlled Pauli P_{left} to the data at the start of the span. At the end it applies $P_{right} = U\,P_{left}\,U^{\dagger}$, where U is the

circuit inside the span. With no error the two cancel and the ancilla reads 0. An error inside the span that anticommutes with the propagated Pauli flips the ancilla, and the run is discarded (post-selection).

Two constraints shape the problem:

- `add_pauli_checks` in qiskit-paulice accepts **Clifford circuits only**.
- A check may cross a **T gate** only where its propagated Pauli is I or Z on that qubit. T commutes with Z, but $T\,X\,T^{\dagger} = (X + Y)/\sqrt{2}$ is not a Pauli, so the check could no longer close.

The quantity to optimise is a trade-off: post-selected fidelity against the fraction of runs kept, at some cost in ancillas, gates and depth.

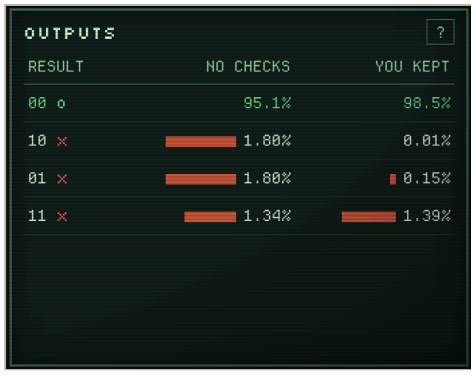


Figure 2. Detection in one picture (puzzle 00). The correct result is 00. With one check, the wrong results 10 and 01 almost vanish from the kept runs, but 11 (two flips) slips through.

2 The game

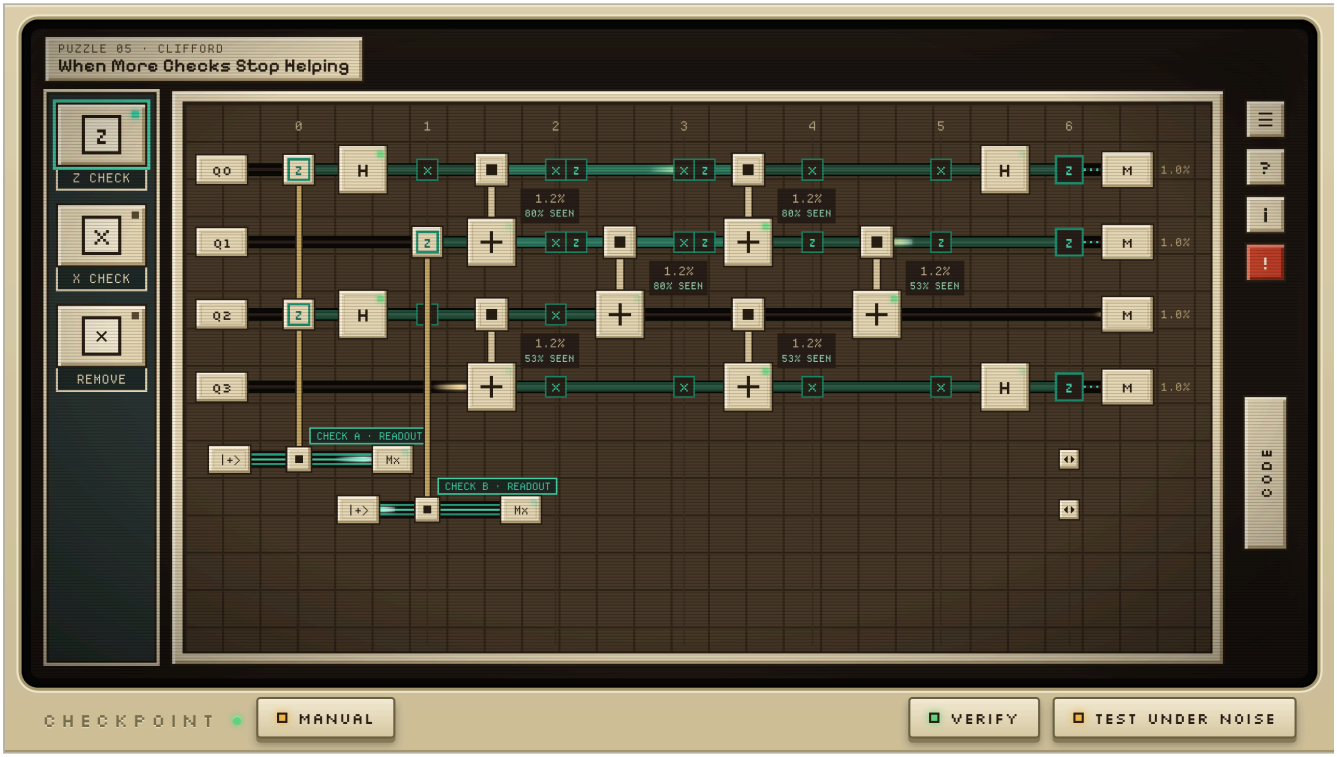


Figure 3. Puzzle 05 with two checks. Check A starts as Z on q0 and q2, check B as Z on q1; both end at the readout. Chips show each check's Pauli at every slot, the wires it watches glow, "SEES" tags name the errors caught there, and each error badge shows the share of that gate's errors the checks would flag.

Each of the 15 puzzles is a small circuit with its own noise model, a budget of ancillas and two targets: a minimum Fidelity and a minimum Yield. The player drags check pieces onto the board and drags each check's end to a later slot. The game propagates the Pauli through the span, draws it at every slot, and shows which errors each gate could produce that the checks would see. **VERIFY** runs the circuit with no noise. **TEST UNDER NOISE** scores it and shows where the layout sits among every layout the engine evaluated. The design follows Zachtronics puzzles: fixed puzzles, several metrics optimised separately, and histograms of all evaluated layouts. These map directly onto the brief's "best trade-off" question.

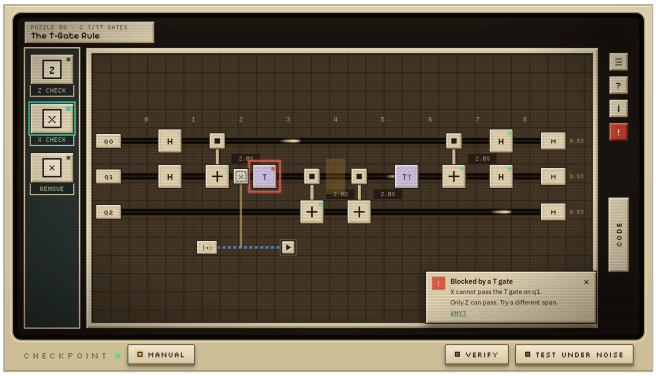


Figure 4. The T rule in play (puzzle 08). An X check on q1 cannot cross the T gate, which is ringed in red; the notice explains why and offers the full reason.

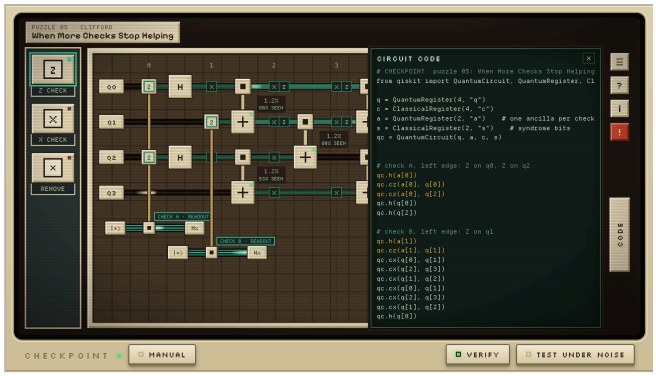


Figure 5. The CODE drawer shows the Qiskit circuit that is actually scored: ancilla registers, controlled-Pauli edges and the payload.

Rules and the physics they represent

GAME ELEMENT	WHAT IT IS
Check piece (Z, X, Y) and its start slot	The Pauli P_{left} and where the check starts in time
Dragging the end	The span; the game computes $P_{right} = U\,P_{left}\,U^{\dagger}$ by propagation (`Pauli.evolve`)
Trail chips and glowing wires	The propagated Pauli at every slot; which wires the check watches
"SEEN" on an error badge	The share of that gate's Pauli errors that anticommute with a check right after it
Ancilla budget	Extra qubits available for checks
Final readout switch ("merged" check)	A Z-only end edge read off the final data measurement instead of controlled gates
Blocked span, red ring on a T	The T rule (Figure 4)
VERIFY	Noiseless run: output unchanged and no check fires (catches clashing checks)
Fidelity / Yield / Cost	Post-selected Hellinger fidelity / kept-shot fraction / ancillas + extra two-qubit gates
Automated reference	qiskit-paulice's own placement with the same budget and noise
CODE drawer	The exact Qiskit circuit being scored, generated from the board (Figure 5)

The puzzles introduce one idea each: choosing the Pauli for the noise (02), ancilla idling on long checks (03), finding a hot gate (04), diminishing returns (05), propagation (06), the readout blind spot (07), the T rule (08), the T-count study (09a to 09d), beating the automated reference (10), and a final puzzle that combines them (11). Each puzzle's lesson is asserted on its computed data by a test, so the text cannot drift from the numbers.



Figure 6. Results for the layout in Figure 3. Fidelity, Yield and Cost each get a histogram of every evaluated layout, with the target as a dashed line (▼ this run, ★ best solved run, ◆ automated reference). The scope plots fidelity against yield with the best trade-offs found drawn as a line; the table compares no checks, the reference and the player.

User manual

The game ships with a 13-page user manual, opened from the MANUAL button on the monitor (Figure 1) or at `/manual`. It is styled after the TIS-100 reference manual: an A4 booklet photocopied in black and white, with a highlighter and pen notes. It explains each screen with numbered callouts, how a check works (propagation, what each Pauli catches, the T rule, final readout), how to read the results, strategy tips and troubleshooting. It prints or saves as an A4 PDF from the browser.

Puzzles 00 and 01 also have guided tours. Every puzzle has a hint and a worked solution behind a hold-to-confirm button; the solution's scores are computed, never stored.

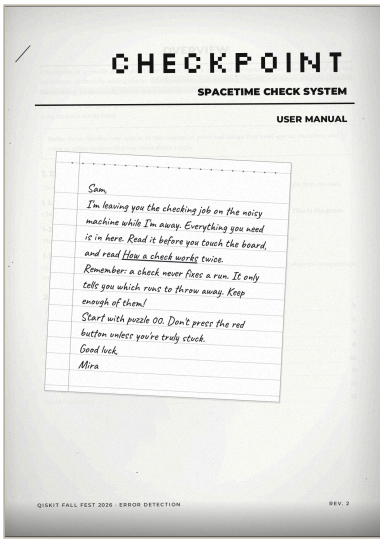


Figure 7. The manual's cover page.

3 Method

Payloads. The experiment uses one 4-qubit circuit family: H on q0 and q2, then CNOTs (0,1), (2,3) and (1,2), with T gates at fixed sites, then the mirror image (U followed by $U^\dagger$). T counts are 0, 2, 4 and 8 (T and $T^\dagger$ both count), and depth is 6, 8, 10 and 10. Because the circuit is a mirror, the ideal output is the single bitstring 0000. This forces T gates to come in T / $T^\dagger$ pairs, which is why the counts are 0, 2, 4, 8 rather than the brief's example 0, 1, 2, 4.

Reference and fidelity measure. The ideal output comes from `StabilizerState` for the Clifford payload and `Statevector` for all payloads; the two are cross-checked on the Clifford one. Fidelity is the **Hellinger fidelity** of the data qubits' output distribution, $F = (\Sigma \sqrt{(p_{ideal}\, p_{kept})})^2$, equal to

`qiskit.quantum_info.hellinger_fidelity` (tested). It is what an experiment that measures the data can see. With a single ideal outcome, F is the probability of the right answer among the kept runs.

Checks. On the Clifford payload: qiskit-paulice `add_pauli_checks` with `cost="gamma"`, `method="windowed"` and a fixed seed, for every number of checks it returns and four target-qubit orders. On every payload, including the Clifford one for comparison: a search over checks that obey the T rule, covering every legal single check plus pairs built from the best singles, within a budget of 2 ancillas.

Noise. The organizers did not announce a common noise model, so three documented models are used. The same parameters drive Aer, the analytic readout and qiskit-paulice's own `NoiseModel`.

- **series**: the game's T-study noise. Two-qubit error 0.015, Z-biased (phase flips dominate), single-qubit 0.0005, idle 0.002 per layer, readout 0.008.
- **uniform**: depolarizing, two-qubit error 0.01 (the analogue of the brief's `NoiseModel(gate_noise=0.01)`), idle 0.001, readout 0.01.
- **backend**: derived from the calibration data of a seeded `GenericBackendV2` (12 qubits, seed 2026, layout [0, 1, 2, 3]).

Scoring is exact. The noise model stays fully in place; only shot sampling is removed. Aer's density-matrix method gives the exact noisy state, readout error is applied analytically to the exact outcome probabilities, and post-selection keeps the outcomes whose syndrome bits are all 0. The same layout therefore always gets the same score, which a puzzle leaderboard needs. A test checks this against 200,000 sampled shots (agreement within 0.005).

Selection rule for "best trade-off". The highest post-selected fidelity with yield ≥ 0.85, ties broken by lower cost. Full Pareto fronts are reported as well.

4 Results of the brief's experiment

Best layout under the selection rule. Added cost is ancillas / extra two-qubit gates / extra depth.

NOISE	T COUNT	F, NO CHECKS	F, BEST (YIELD)	GAIN	ADDED COST	LEGAL SINGLE CHECKS
series (Z-biased)	0	0.896	**0.990** (0.853)	+0.094	2 / 4 / 3	1760
	2	0.888	0.951 (0.893)	+0.063	2 / 3 / 1	1407
	4	0.877	0.910 (0.874)	+0.033	2 / 4 / 1	1186
	8	0.882	0.909 (0.882)	+0.027	2 / 4 / 3	866
uniform	0	0.907	**0.983** (0.873)	+0.076	2 / 4 / 3	1760
	2	0.903	0.968 (0.892)	+0.065	2 / 3 / 1	1407
	4	0.897	0.952 (0.900)	+0.055	2 / 3 / 1	1186
	8	0.898	0.953 (0.901)	+0.055	2 / 3 / 1	866
backend	0	0.973	**0.997** (0.959)	+0.024	2 / 4 / 3	1760
	2	0.966	0.990 (0.967)	+0.024	2 / 3 / 1	1407
	4	0.955	0.982 (0.944)	+0.027	2 / 5 / 3	1186
	8	0.962	0.984 (0.971)	+0.022	2 / 3 / 0	866

qiskit-paulice on the Clifford payload, best entry under the same rule across all four target orders:

NOISE	QISKIT-PAULICE: F (YIELD), COST	BEST SEARCHED LAYOUT: F (YIELD), COST
series	0.946 (0.863), 6	0.990 (0.853), 6
uniform	0.933 (0.917), 6	0.983 (0.873), 6
backend	0.983 (0.969), 6	0.997 (0.959), 6

Full tables, including every check count and target order, are in `experiments/results/<noise>/summary.md`.

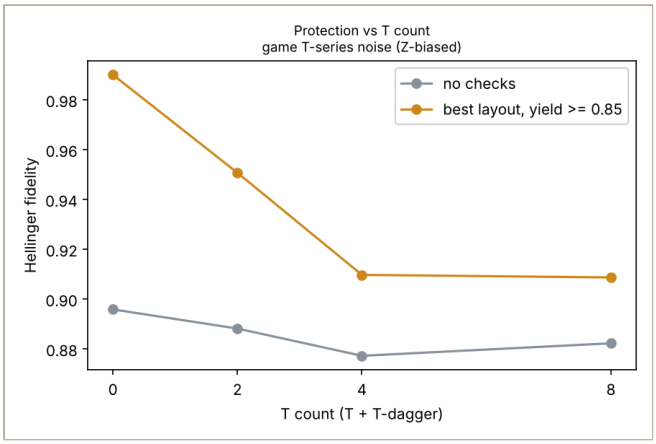


Figure 8. Series noise: fidelity with no checks and with the best layout at yield ≥ 0.85, against T count.

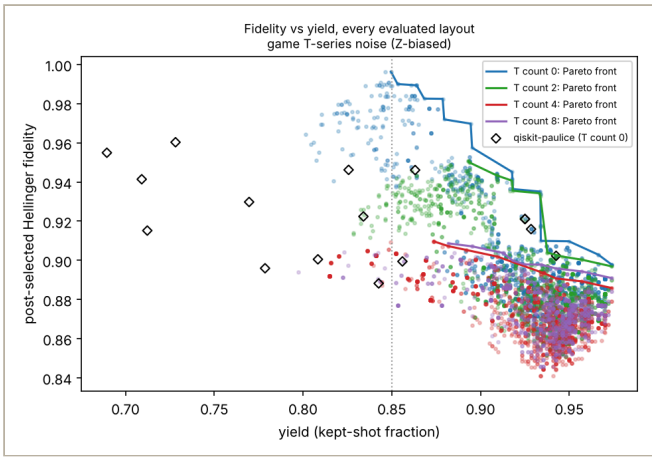


Figure 9. Series noise: every evaluated layout, the Pareto front for each T count, and qiskit-paulice's layouts (diamonds). The dotted line is the yield floor.

The best trade-off and why it wins

In all three noise models the winning Clifford layout has the same shape: **two checks that span the whole circuit and end at the readout, each starting on two qubits** (for example Z on q0 and q2 plus Z on q1 and q3). It wins for four reasons:

1. **Coverage.** A check that spans the whole circuit watches every gate, so almost every error that reaches the output is visible to at least one check.
2. **Low cost.** Its end edge is Z-only at the readout, so it is merged: the syndrome is read from the data measurement itself. It costs only its start-edge gates (2 two-qubit gates per check, cost 6 in total).
3. **No ancilla idling.** A merged check's ancilla can be measured right after its start edge, so it never waits through the span and collects no idle noise.
4. **Readout flips caught.** Reading the parity from the data measurement also catches a single readout flip on those qubits, which a gate-based check cannot.

The experiment's budget is two ancillas. Starting each check on two qubits lets the pair watch all four data qubits (or the three that matter most under the noise).

5 Findings

1. T gates shrink the space of legal checks, and how much that costs depends on the noise.

- Legal single checks fall from 1760 to 866 as the T count goes from 0 to 8.
- Under phase-flip-dominated noise the gain from checks collapses from +0.094 to +0.027. Phase flips are caught only by checks that are X-type at that point, and X-type checks are exactly what a T gate forbids. The winners at T count 4 and 8 rely on X checks that either avoid the T qubits or start after the last T gate.
- Under depolarizing noise the gain falls more gently (+0.076 to +0.055), because Z-type checks, which T gates allow, still catch two of the three Pauli error types (X and Y). Under the backend-derived noise it is flat (+0.022 to +0.027).
- So "T gates hurt error detection" is true, but the size of the effect is a property of the noise bias, not of the T count alone. T count 4 and 8 share a depth and score alike: part of the trend is depth, not T count.

2. More checks stop helping, and can make things worse.

- In puzzle 05 the best fidelity with 1, 2, 3 and 4 checks is 0.965, 0.989, 0.990 and 0.990, while the yield of those layouts falls 0.919, 0.882, 0.855, 0.800. The fourth check adds nothing and costs a twentieth of the runs.
- With qiskit-paulice under backend noise (target order 0-1-2-3), fidelity goes 0.975, 0.964, 0.948 for 1, 2, 3 checks. That is *below* the unprotected 0.973, because each check's own gates add errors.

3. Where a check ends matters as much as where it starts.

- In puzzle 07 the gates are good but readout is poor (4% per qubit). The unprotected circuit scores 0.868.
- The best gate-based check reaches only 0.871: it finishes before the measurement and cannot see a readout flip.
- The best merged check reaches 0.976 (yield 0.846). This is still detection: it post-selects on the data's measured parity.

4. The automated reference optimises a different objective, and depends on its inputs.

- Under our rule the searched layouts beat qiskit-paulice in every noise model at the same cost of 6. This is a comparison of objectives, not of skill: qiskit-paulice minimises its gamma cost (the weight of undetectable errors) and does not see a yield floor, while our search was ranked by the metric it is judged on.
- paulice's result depends on the order of `target_qubits`. Under series noise its two-check layout scores 0.922 with order 0-1-2-3 and 0.946 with order 2-0-3-1.
- During development we also saw ties in gamma broken nondeterministically: on puzzle 10, 1 run in 6 returned a different check set with identical gamma, scoring 0.925 against 0.918 fidelity. Pinning `PYTHONHASHSEED` did not help. The game therefore caches paulice results on disk, keyed by circuit, noise, seed and library version, so the reference is stable for every player.

5. The fidelity measure needs a peaked ideal output.

- While calibrating puzzles, payloads whose ideal output spread over 8 to 32 bitstrings scored a Hellinger fidelity near 0.99 or more with no protection at all, so checks looked useless.
- Hellinger fidelity of a broad distribution barely reacts to noise. Mirror circuits (one correct outcome) and GHZ-like circuits (two to eight) fixed this; every payload in the game and the experiment is one of these. This is worth knowing for anyone benchmarking detection with distribution-level metrics.

6 Limitations

- **Noise model.** The noise is a Pauli-channel model: no coherent errors, crosstalk or leakage. The backend path is a depolarizing approximation of a generic backend's calibration, without routing. Puzzle noise is hand-set per

puzzle so that each lesson shows up; the experiment's models are stated above.

- **Search, not proof.** "Best found" layouts come from a search (all legal single checks, pairs of the best, and a seeded random sample in the game). They are strong heuristic optima, not proven ones, and the game says so on screen.
- **Small circuits.** Exact density-matrix scoring limits circuits to a handful of qubits plus ancillas. That suits a puzzle, not a large-scale study.
- **Even T counts.** T counts are 0, 2, 4, 8 because mirror circuits pair T with $T^\dagger$.
- **Cached reference.** The automated reference is cached; deleting `data/paulice_cache/` regenerates it, possibly with different tie-breaks.

7 Reproducing

```
pip install -r requirements.txt          # Python 3.12 or newer
python -m pytest -q                      # 79 tests
python experiments/brief_experiment.py   # about 2.5 minutes
python -m checkpoint.server              # http://127.0.0.1:8000
```

The executed notebook `experiments/brief_experiment.ipynb` reproduces the experiment, and `docs/level_design.md` lists every puzzle's numbers.

Qiskit used. qiskit-paulice: `add_pauli_checks`, `NoiseModel`, `CheckedCircuit.check_support`. qiskit-aer: `AerSimulator` with the density-matrix method and a Pauli-channel noise model. `qiskit.quantum_info`: `Pauli.evolve` for propagation, `StabilizerState` and `Statevector` for references, `Clifford` to resynthesize paulice's single-qubit chains, and `state_fidelity`; the game's Hellinger fidelity is tested against `hellinger_fidelity`. `GenericBackendV2` for backend-derived noise, and `qpy` for caching circuits.